# Quantum information and quantum computation homework 2

Zhouyue Qian 3220104074 *

*(Information and Computational Science Class 2201), Zhejiang University*

Due time: November 19, 2024

## 1 3SAT to SAT

### Proof

We aim to prove that **3SAT** is reducible to **SAT**. The following steps outline the proof:

1. **Definition of 3SAT**: 3SAT is a restricted version of the SAT problem where each clause contains exactly three literals. For example:

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_4 \vee x_5 \vee x_6).$$

2. **Definition of SAT**: SAT is the general Boolean satisfiability problem, where each clause can have any number of literals.

3. **Goal**: To show that 3SAT is reducible to SAT, we need to transform any instance of 3SAT into an equivalent SAT formula in polynomial time.

4. **Reduction**: The transformation from 3SAT to SAT is trivial since every 3SAT formula is already in conjunctive normal form (CNF), which is a valid form for SAT. Since SAT allows clauses of any length, a formula with clauses containing exactly three literals is directly interpretable as a SAT formula without any modifications.

5. **Preservation of Satisfiability**: A 3SAT formula $\phi$ and its corresponding SAT formula $\phi'$ are identical. Therefore: - If $\phi$ is satisfiable in 3SAT, then $\phi'$ is satisfiable in SAT. - If $\phi$ is unsatisfiable in 3SAT, then $\phi'$ is unsatisfiable in SAT.

6. **Complexity**: The transformation involves no structural changes and is thus computable in linear time relative to the size of the input formula.

7. **Conclusion**: Since 3SAT is a special case of SAT, every instance of 3SAT is reducible to SAT in polynomial time.

## 2 SAT to 3SAT

### Introduction

To prove the correctness of the reduction from SAT to 3SAT, we need to show that the transformation preserves satisfiability. Specifically, for each clause $C$ in the original SAT formula $\phi$, the replacement clauses in the 3SAT formula $\phi'$ are constructed such that $\phi$ is satisfiable if and only if $\phi'$ is satisfiable.

*Electronic address: 3220104074@edu.zju.cn

The reduction handles clauses of different lengths separately:

- Clauses with one literal (Case 1)
- Clauses with two literals (Case 2)
- Clauses with exactly three literals (Case 3)
- Clauses with more than three literals (Case 4)

We will go through each case, explain the replacement, and prove that the satisfiability is preserved.

## Case 1: Clause with One Literal

**Original Clause:**
Let $C = \ell$, where $\ell$ is a literal (either $x_i$ or $\neg x_i$).
**Replacement Clauses:**
Introduce two new variables $z_1$ and $z_2$, and replace $C$ with the following four clauses:

$$(\ell \vee z_1 \vee z_2),$$
$$(\ell \vee \neg z_1 \vee z_2),$$
$$(\ell \vee z_1 \vee \neg z_2),$$
$$(\ell \vee \neg z_1 \vee \neg z_2).$$

**Proof of Equivalence:**
The logical AND of these four clauses is equivalent to $\ell$.

- If $\ell$ is **True**, all four clauses are **True** regardless of the values of $z_1$ and $z_2$.
- If $\ell$ is **False**, we need to satisfy all clauses with $\ell = \text{False}$. The clauses become:

$$(\text{False} \vee z_1 \vee z_2),$$
$$(\text{False} \vee \neg z_1 \vee z_2),$$
$$(\text{False} \vee z_1 \vee \neg z_2),$$
$$(\text{False} \vee \neg z_1 \vee \neg z_2).$$

These clauses represent all possible combinations of $z_1$ and $z_2$, meaning at least one clause will be **False** regardless of how $z_1$ and $z_2$ are assigned.

**Conclusion:** The replacement preserves satisfiability. The new formula is satisfiable if and only if the original clause $\ell$ is satisfiable.

## Case 2: Clause with Two Literals

**Original Clause:**
Let $C = \ell_1 \vee \ell_2$, where $\ell_1$ and $\ell_2$ are literals.
**Replacement Clauses:**
Introduce one new variable $z_1$, and replace $C$ with the following two clauses:

$$(\ell_1 \vee \ell_2 \vee z_1),$$
$$(\ell_1 \vee \ell_2 \vee \neg z_1).$$

**Proof of Equivalence:**
The logical AND of these two clauses is equivalent to $\ell_1 \vee \ell_2$.

- If $\ell_1 \vee \ell_2$ is **True**, both clauses are **True** regardless of $z_1$.
- If $\ell_1 \vee \ell_2$ is **False**, then $\ell_1$ and $\ell_2$ are both **False**. The clauses become:

$$(\text{False} \vee z_1),$$
$$(\text{False} \vee \neg z_1).$$

This reduces to $z_1$ and $\neg z_1$, which cannot both be **True** simultaneously.

**Conclusion:** The replacement preserves satisfiability. The new formula is satisfiable if and only if the original clause $C$ is satisfiable.

## Case 3: Clause with Exactly Three Literals

**Action:** Leave the clause unchanged.
**Conclusion:** No change is necessary, and satisfiability is trivially preserved.

## Case 4: Clause with More Than Three Literals

**Original Clause:**
Let $C = \ell_1 \vee \ell_2 \vee \ell_3 \vee \ldots \vee \ell_k$, where $k > 3$ and each $\ell_i$ is a literal.
**Replacement Clauses:**
Introduce $k-3$ new variables $z_1, z_2, \ldots, z_{k-3}$, and replace $C$ with the following $k-2$ clauses:

$$(\ell_1 \vee \ell_2 \vee z_1),$$
$$(\ell_3 \vee \neg z_1 \vee z_2),$$
$$(\ell_4 \vee \neg z_2 \vee z_3),$$
$$\vdots$$
$$(\ell_{k-2} \vee \neg z_{k-4} \vee z_{k-3}),$$
$$(\ell_{k-1} \vee \ell_k \vee \neg z_{k-3}).$$

**Objective:** Prove the following statements:

1. **Statement 1**:If there exists an assignment to $x_1, \ldots, x_n$ such that $C$ is **True**, then there exists an assignment to $z_1, \ldots, z_{k-3}$ such that all replacement clauses are **True**.
2. **Statement 2**:If, for a given assignment to $x_1, \ldots, x_n$, $C$ is **False**, then for any assignment to $z_1, \ldots, z_{k-3}$, at least one of the replacement clauses is **False**.

## Proof of Statement 1:

**Assumption:** There exists an assignment to $x_1, \ldots, x_n$ such that $C = \ell_1 \vee \ell_2 \vee \ldots \vee \ell_k$ is **True**.
**Objective:** Find an assignment to $z_1, \ldots, z_{k-3}$ such that all replacement clauses are **True**.
**Proof:** Since $C$ is **True**, at least one of the literals $\ell_1, \ldots, \ell_k$ is **True**.
**Case A:** If any of $\ell_1$ or $\ell_2$ is **True**.

- Then, the first clause $(\ell_1 \vee \ell_2 \vee z_1)$ is **True** regardless of $z_1$.
- Assign $z_1$ arbitrarily.
- Proceed recursively with the rest of the clauses.

**Case B:** If $\ell_1$ and $\ell_2$ are **False**, but $\ell_i$ is **True** for some $i \geq 3$.
Let $j$ be the smallest index $\geq 3$ such that $\ell_j$ is **True**.
Assign $z_1, z_2, \ldots, z_{j-3}$ such that:

- For $t = 1$ to $j - 3$, set $z_t = \text{False}$.
- This makes $\neg z_{t-1} = \text{True}$ (for $t \geq 2$), satisfying the clauses.

The clause $(\ell_j \vee \neg z_{j-3} \vee z_{j-2})$ (if $j < k$) or $(\ell_{k-1} \vee \ell_k \vee \neg z_{k-3})$ (if $j = k$) will be satisfied because $\ell_j$ is **True**.
For $t = j - 2$ to $k - 3$, assign $z_t$ arbitrarily (if any).

**Example for Clarity**

Suppose $k = 6$, literals are $\ell_1, \ell_2, \ell_3, \ell_4, \ell_5, \ell_6$.
Suppose $\ell_4$ is **True**, and $\ell_1, \ell_2, \ell_3$ are **False**.

- Assign $z_1 = \text{False}$, so $\neg z_1 = \text{True}$.
- The second clause $(\ell_3 \vee \neg z_1 \vee z_2)$ is satisfied since $\neg z_1 = \text{True}$.
- Assign $z_2 = \text{False}$, so $\neg z_2 = \text{True}$.
- The third clause $(\ell_4 \vee \neg z_2 \vee z_3)$ is satisfied because $\ell_4 = \text{True}$.

**Conclusion:** By appropriately assigning $z_i$, we can ensure all clauses are **True** when $C$ is **True**.

## Proof of Statement 2

Since $C$ is **False**, all $\ell_i$ are **False**.
We will show that the clauses form a chain of dependencies that cannot be satisfied when all $\ell_i$ are **False**.

- **First Clause:** $(\ell_1 \vee \ell_2 \vee z_1)$
  - Since $\ell_1$ and $\ell_2$ are **False**, the clause reduces to $(z_1)$.
- **Second Clause:** $(\ell_3 \vee \neg z_1 \vee z_2)$
  - Since $\ell_3$ is **False**, the clause reduces to $(\neg z_1 \vee z_2)$.
- **Third Clause:** $(\ell_4 \vee \neg z_2 \vee z_3)$
  - Since $\ell_4$ is **False**, the clause reduces to $(\neg z_2 \vee z_3)$.
- $\vdots$
- **Last Clause:** $(\ell_{k-1} \vee \ell_k \vee \neg z_{k-3})$
  - Since $\ell_{k-1}$ and $\ell_k$ are **False**, the clause reduces to $(\neg z_{k-3})$.

Now, we can derive a contradiction:

- From the first clause, $z_1$ must be **True**.
- From the second clause, since $z_1 = \text{True}$, $\neg z_1 = \text{False}$, so $z_2$ must be **True**.
- From the third clause, $z_2 = \text{True}$ implies $\neg z_2 = \text{False}$, so $z_3$ must be **True**.
- Continue this pattern up to $z_{k-3}$.
- From the last clause, $\neg z_{k-3}$ must be **True**, so $z_{k-3}$ must be **False**.

**Contradiction:** $z_{k-3} = \text{True}$ and $z_{k-3} = \text{False}$.

**Conclusion:** It's impossible to assign values to $z_1, \ldots, z_{k-3}$ to satisfy all clauses when all $\ell_i$ are **False**. Therefore, the conjunction of the replacement clauses is **False**.

## Overall Conclusion

The reduction from SAT to 3SAT preserves satisfiability for each clause:

- If a clause $C$ is **True** under some assignment, the replacement clauses can be satisfied by extending the assignment to the new variables.
- If a clause $C$ is **False** under some assignment, the replacement clauses cannot be satisfied, regardless of how the new variables are assigned.

By applying this transformation to each clause in $\phi$, we ensure that the overall formula $\phi'$ is satisfiable if and only if $\phi$ is satisfiable.

Now the proof is completed.□